SGT UNIVERSITY

GURUGRAM, HARYANA

SCHOOL OF ENGINEERING AND TECHNOLOGY

Department of Computer Science and Engineering (AI/ML)

DATA STRUCTURES AND ALGORITHMS

EVENT REGISTRATION PORTAL

A Python console application demonstrating linked lists, queues, and stacks

PROJECT DETAILS	INFORMATION
Submitted by	Harshita | Registration No. 251302278
	Khushi | Registration No. 251302279
Programme / Section	B.Tech CSE (AI/ML) | Section D
Academic session	2026-27
Submitted to	Mr. Vivek Aggarwal
Course	Data Structures and Algorithms

Academic Project Report

Certificate

This is to certify that the project report entitled “Event Registration Portal” is a bona fide academic work prepared by Harshita (Registration No. 251302278) and Khushi (Registration No. 251302279), students of B.Tech Computer Science and Engineering (AI/ML), Section D, SGT University, during academic session 2026-27.

The work has been completed as part of the Data Structures and Algorithms course under the guidance of Mr. Vivek Aggarwal. It presents a Python implementation using linked lists, a FIFO queue, and a LIFO stack.

Faculty Guide: Mr. Vivek Aggarwal

Signature: _____ Date: _____

Department / University Seal: _____

Acknowledgement

We thank Mr. Vivek Aggarwal for his guidance and feedback while preparing this project. We also thank the Department of Computer Science and Engineering (AI/ML), SGT University, for the academic environment and resources that supported this work. Finally, we thank our classmates and families for their encouragement.

Harshita and Khushi
B.Tech CSE (AI/ML), Section D

Declaration

We declare that this report, titled “Event Registration Portal,” has been prepared by us for academic learning. The program and explanations are presented as a student project. External technical references are listed in the bibliography.

Harshita: _____ Khushi: _____ Date: _____

Abstract

The Event Registration Portal is a small Python console application for maintaining events and processing attendee registration requests. It demonstrates core structures through practical workflows: a singly linked list stores events and attendees, a first-in, first-out queue holds requests until they are processed, and a last-in, first-out stack records recent actions. The program supports adding and listing events, queuing requests, processing the oldest request, viewing attendees, and reviewing action history. It uses only the Python standard library and stores data in memory for one run. The report explains requirements, design, algorithms, pseudocode, complete source code, sample output, complexity, and test cases.

Table of Contents

SECTION	CONTENTS
Front matter	Certificate; Acknowledgement; Declaration; Abstract
1	Introduction
2	Problem statement, objectives and scope
3	Requirements and assumptions
4	Data structures and choices
5	System design and workflow
6	Algorithms and pseudocode
7	Complete Python source code
8	Sample console outputs
9	Complexity analysis
10	Testing and test cases
11	Applications, limitations and future scope
12	Conclusion
Appendix A	How to run
References	Technical references

Page numbers appear in the footer. This section index follows the report order.

1. Introduction

Events such as workshops, seminars, club activities, and college fests need a way to keep event details and attendee lists. If requests arrive over time, an organizer also needs to process them in arrival order. This project builds a small menu-based portal in Python to demonstrate how data structures can model these tasks. It is an educational prototype, not a production registration service.

1.1 Project overview

ITEM	DESCRIPTION
Name	Event Registration Portal
Language	Python 3
Interface	Console menu
Persistence	In memory during one run
Structures	Singly linked list, linked queue, linked stack
Use	Create events and process registration requests

2. Problem Statement, Objectives and Scope

2.1 Problem statement

A basic event organizer needs to add events, accept attendee requests, process requests in order, and inspect who registered. This project uses structures whose behavior directly matches each task.

2.2 Objectives

Build a menu-based Python portal for a small event list.

Store events and attendee names in linked nodes.

Queue requests in FIFO order.

Record outcomes in a stack and show newest actions first.

Explain algorithms and costs using Big-O notation.

Check normal, boundary, and invalid-input cases.

2.3 Scope

The operator can create named events, submit attendee requests, process requests, and view events, attendees, the queue, and action history. Event names are unique case-insensitively; each event has a capacity. Unknown events, duplicates, and full events are rejected during processing. The program is single-user, in-memory, and has no accounts, database, network access, payments, or email.

3. Requirements and Assumptions

3.1 Functional requirements

Add an event with a non-empty name and positive capacity.

Display events, capacities, and attendee counts.

Accept event and attendee names as a request.

Process the oldest queued request.

Register valid requests or report why they were rejected.

Show attendees, pending requests, and recent actions.

3.2 Non-functional requirements

Readable Python using only standard-library features.

Input validation for menu choices, capacity, and required text.

Predictable output and small modular classes.

3.3 Assumptions

One operator uses one terminal session.

Event and attendee names are compared without case.

Queue is unbounded in this teaching version.

Requests are validated at processing time.

The stack retains at most 10 recent outcomes.

4. Data Structures and Design Choices

4.1 Singly linked list

Each node stores a value and a reference to the next node. EventNode records the name, capacity, attendee-list head, count, and next event. AttendeeNode stores one name and its next link. Event and attendee searches walk nodes one at a time. Tail insertion and duplicate checks therefore take linear time. This makes linked-list traversal visible and easy to trace.

4.2 Queue for requests

A queue follows FIFO (First In, First Out). Front and rear references allow enqueue at the rear and dequeue from the front in O(1) time. Thus requests are processed in submission order.

4.3 Stack for recent actions

A stack follows LIFO (Last In, First Out). New actions are pushed at the top, so the newest appears first. The program bounds history to 10 items; when full, it removes the oldest item.

4.4 Choices and trade-offs

STRUCTURE	USE	BENEFIT	TRADE-OFF
Linked list	Events, attendees	Node links and traversal	Lookup is O(n)
Queue	Requests	Fair FIFO order; O(1) add/remove	No arbitrary removal
Stack	Action history	Newest action first	Only 10 outcomes retained

5. System Design and Workflow

5.1 Components

EventNode and AttendeeNode hold linked records.

EventList adds, searches, lists events, and manages attendees.

RequestQueue stores registration requests.

ActionStack keeps a bounded recent history.

Portal validates menu input and coordinates operations.

5.2 Registration workflow

Operator enters event and attendee names.

Program adds the request to the queue rear.

Operator chooses Process next request.

Program removes the front request.

It searches for the event and checks duplicate and capacity rules.

A valid attendee is appended; otherwise, a rejection is reported.

The result is pushed to the action stack.

5.3 Flowchart

```
START
  |
Display menu and read choice
  |
Choice 0? -- yes --> END
  | no
Perform chosen operation
  |
Registration request --> enqueue
  |
Process request --> dequeue oldest
  |
Find event; check duplicate and capacity
  |
Register attendee OR show rejection
  |
Push outcome onto action stack
  |
Return to menu
```

Figure 1. Text flowchart for menu and request processing.

6. Algorithms and Pseudocode

6.1 Add event

```
ADD_EVENT(name, capacity)
  IF name empty OR capacity <= 0: report invalid; RETURN
  IF FIND_EVENT(name) exists: report duplicate; RETURN
  CREATE event node
  IF list empty: head = node
  ELSE walk to last node; last.next = node
  PUSH success action
```

6.2 Enqueue a request

```
ENQUEUE(event_name, attendee_name)
  IF either name empty: report invalid; RETURN
  CREATE request node
  IF queue empty: front = rear = node
  ELSE rear.next = node; rear = node
```

6.3 Process request

```
PROCESS_NEXT()
  IF queue empty: report no requests; RETURN
  request = REMOVE_FRONT()
```

```
  event = FIND_EVENT(request.event_name)
  IF event missing: result = unknown event
  ELSE IF attendee already exists: result = duplicate
  ELSE IF event full: result = full
  ELSE append attendee; increment count; result = registered
  PUSH result on action stack; report result
```

6.4 Push bounded action history

```
PUSH_ACTION(message)
  PUSH message at stack top
  IF size > 10: unlink bottom node; decrement size
```

7. Complete Python Source Code

Save as event_registration_portal.py. The program runs with Python 3 and no third-party packages.

```
class AttendeeNode:
    def __init__(self, name): self.name, self.next = name, None
class EventNode:
    def __init__(self, name, capacity):
        self.name, self.capacity = name, capacity
        self.attendee_head, self.attendee_count, self.next = None, 0, None
class EventList:
    def __init__(self): self.head = None
    def find(self, name):
        cur = self.head
        while cur:
            if cur.name.casefold() == name.casefold(): return cur
            cur = cur.next
        return None
    def add(self, name, capacity):
        name = name.strip()
        if not name or capacity <= 0: return False, "Event name and positive capacity are
required."
        if self.find(name): return False, "An event with that name already exists."
        node = EventNode(name, capacity)
        if not self.head: self.head = node
        else:
            cur = self.head
            while cur.next: cur = cur.next
            cur.next = node
        return True, f"Event '{name}' added."
    def list_events(self):
        result, cur = [], self.head
        while cur: result.append(cur); cur = cur.next
        return result
    def add_attendee(self, event, name):
        name = name.strip()
        if not name: return False, "Attendee name is required."
        cur = event.attendee_head
        while cur:
```

```
            if cur.name.casefold() == name.casefold(): return False, "That attendee is already
registered for this event."
            cur = cur.next
        if event.attendee_count >= event.capacity: return False, "The event is full."
        node = AttendeeNode(name)
        if not event.attendee_head: event.attendee_head = node
        else:
            cur = event.attendee_head
            while cur.next: cur = cur.next
            cur.next = node
        event.attendee_count += 1
        return True, f"'{name}' registered for '{event.name}'."
    def attendees(self, event):
        names, cur = [], event.attendee_head
        while cur: names.append(cur.name); cur = cur.next
        return names
class RequestNode:
    def __init__(self, event, attendee):
        self.event, self.attendee, self.next = event, attendee, None
class RequestQueue:
    def __init__(self): self.front = self.rear = None
    def enqueue(self, event, attendee):
        node = RequestNode(event.strip(), attendee.strip())
        if not self.rear: self.front = self.rear = node
        else: self.rear.next = node; self.rear = node
    def dequeue(self):
        if not self.front: return None
        node = self.front; self.front = node.next
        if not self.front: self.rear = None
        return node
    def items(self):
        result, cur = [], self.front
        while cur: result.append((cur.event, cur.attendee)); cur = cur.next
        return result
class ActionNode:
    def __init__(self, message): self.message, self.next = message, None
class ActionStack:
    def __init__(self, limit=10): self.top, self.size, self.limit = None, 0, limit
    def push(self, message):
        node = ActionNode(message); node.next = self.top; self.top = node; self.size += 1
        if self.size > self.limit:
            cur = self.top
            while cur.next and cur.next.next: cur = cur.next
            if cur.next: cur.next = None; self.size -= 1
    def items(self):
        result, cur = [], self.top
        while cur: result.append(cur.message); cur = cur.next
        return result
class Portal:
```

```
    def __init__(self):
        self.events, self.requests, self.actions = EventList(), RequestQueue(), ActionStack(10)
    def add_event(self, name, capacity):
        ok, message = self.events.add(name, capacity)
        if ok: self.actions.push(message)
        return message
    def queue_registration(self, event, attendee):
        if not event.strip() or not attendee.strip(): return "Event name and attendee name are
required."
        self.requests.enqueue(event, attendee)
        return f"Request queued for {attendee.strip()} -> {event.strip()}."
    def process_next(self):
        req = self.requests.dequeue()
        if not req: return "No pending registration requests."
        event = self.events.find(req.event)
        if not event: message = f"Rejected: event '{req.event}' was not found."
        else: _, message = self.events.add_attendee(event, req.attendee)
        self.actions.push(message); return message
    def show_events(self):
        events = self.events.list_events()
        if not events: print("No events have been added yet."); return
        for e in events: print(f"- {e.name}: {e.attendee_count}/{e.capacity} registered")
    def show_attendees(self, name):
        event = self.events.find(name)
        if not event: print("Event not found."); return
        names = self.events.attendees(event)
        print(f"Attendees for {event.name} ({len(names)}/{event.capacity}):")
        if not names: print("  (none)")
        for i, person in enumerate(names, 1): print(f"  {i}. {person}")
    def run(self):
        while True:
            print("\n=== Event Registration Portal ===")
            print("1. Add event\n2. List events\n3. Queue registration request")
            print("4. Process next request\n5. Show attendees\n6. Show pending requests")
            print("7. Show recent actions\n0. Exit")
            choice = input("Choose an option: ").strip()
            if choice == "0": print("Goodbye."); break
            elif choice == "1":
                name = input("Event name: ").strip()
                try: print(self.add_event(name, int(input("Capacity: ").strip())))
                except ValueError: print("Capacity must be a whole number.")
            elif choice == "2": self.show_events()
            elif choice == "3": print(self.queue_registration(input("Event name: "),
input("Attendee name: ")))
            elif choice == "4": print(self.process_next())
            elif choice == "5": self.show_attendees(input("Event name: ").strip())
            elif choice == "6":
                pending = self.requests.items()
                if not pending: print("No pending requests.")
```

```
                for i, (event, attendee) in enumerate(pending, 1): print(f"{i}. {attendee} ->
{event}")
            elif choice == "7":
                actions = self.actions.items()
                if not actions: print("No recent actions.")
                for i, action in enumerate(actions, 1): print(f"{i}. {action}")
            else: print("Invalid menu choice. Enter a number from 0 to 7.")
if __name__ == "__main__": Portal().run()
```

8. Sample Console Outputs

These examples show messages produced by the implementation.

8.1 Add event and queue requests

```
Choose an option: 1
Event name: AI Workshop
Capacity: 2
Event 'AI Workshop' added.
Choose an option: 3
Event name: AI Workshop
Attendee name: Asha
Request queued for Asha -> AI Workshop.
Choose an option: 3
Event name: AI Workshop
Attendee name: Rohan
Request queued for Rohan -> AI Workshop.
```

8.2 Process FIFO requests

```
Choose an option: 4
'Asha' registered for 'AI Workshop'.
Choose an option: 4
'Rohan' registered for 'AI Workshop'.
Choose an option: 5
Event name: AI Workshop
Attendees for AI Workshop (2/2):
  1. Asha
  2. Rohan
Choose an option: 4
No pending registration requests.
```

8.3 Rejections and stack history

```
Choose an option: 3
Event name: AI Workshop
Attendee name: Mina
Request queued for Mina -> AI Workshop.
Choose an option: 4
The event is full.
Choose an option: 3
Event name: Unknown Meetup
Attendee name: Dev
Request queued for Dev -> Unknown Meetup.
Choose an option: 4
Rejected: event 'Unknown Meetup' was not found.
Choose an option: 7
1. Rejected: event 'Unknown Meetup' was not found.
2. The event is full.
3. 'Rohan' registered for 'AI Workshop'.
4. 'Asha' registered for 'AI Workshop'.
5. Event 'AI Workshop' added.
```

New actions appear first (LIFO); requests are processed oldest first (FIFO).

9. Complexity Analysis

Let E be the number of events, A the attendees in one event, Q pending requests, and H the history limit of 10.

OPERATION	TIME	SPACE	REASON
Find event	O(E)	O(1)	Sequential traversal
Add event	O(E)	O(1)	Duplicate check and tail traversal
List events	O(E)	O(E) output	Visits all events
Check/add attendee	O(A)	O(1)	Duplicate check and tail append
Enqueue / dequeue	O(1) each	O(1) per node	Rear link / front removal
List queue	O(Q)	O(Q) output	Visits requests
Push action	O(1), O(H) on trim	O(1)	Push plus bounded trim
Show actions	O(H)	O(H) output	Visits retained history

Overall storage is O(E + total attendees + Q + H). Event and attendee lookup use linear traversal. The fixed history limit keeps action storage bounded.

10. Testing and Test Cases

Expected-result cases for the supplied program are listed below. Students should run them locally and record actual observations if required by the instructor.

ID	TEST	EXPECTED RESULT
T01	Add AI Workshop capacity 2; list.	Appears as 0/2.
T02	Add “ai workshop” again.	Duplicate rejected case-insensitively.
T03	Queue Asha then Rohan; process twice.	Asha then Rohan register.
T04	Process empty queue.	No pending registration requests.
T05	Request unknown event; process.	Rejected as event not found.
T06	Fill event and process another request.	Extra registration rejected as full.
T07	Register Asha then “asha”.	Second registration rejected as duplicate.
T08	Enter non-integer capacity.	Error message; menu continues.
T09	Enter unsupported menu choice.	Invalid choice message.
T10	Log more than 10 outcomes.	Only 10 newest actions shown.

11. Applications, Limitations and Future Scope

11.1 Applications

Classroom demonstration of linked structures and traversal.

Single-session sign-up prototype for a small club or lab.

Starting point for studying FIFO processing and LIFO history.

11.2 Limitations

Data disappears when the program exits.

No authentication, multiple users, or network support.

Linear searches are not suited to large datasets.

No cancellation, waitlist, queue limit, persistence, or export.

11.3 Future scope

Save and load records from files or a database.

Add authenticated web or graphical interface.

Add cancellation, waitlists, and capacity-aware request handling.

Add dates, venues, confirmation numbers, and reports.

Add a dictionary index and compare lookup costs.

Create automated unit tests and stricter validation.

12. Conclusion

The Event Registration Portal shows how data structures can model practical behavior. Linked lists store events and attendees, a queue preserves request arrival order, and a stack displays recent actions from newest to oldest. The menu program handles common outcomes such as duplicates, full events, missing events, empty queues, and invalid input. The complexity analysis also shows where linear traversal is appropriate for a small teaching example and where a larger system would need indexing, persistence, security, and stronger validation.

Appendix A. How to Run

Install Python 3.

Copy Section 7 into event_registration_portal.py.

Open a terminal in that folder.

Run python event_registration_portal.py (or python3 event_registration_portal.py).

Choose options 1-7 and enter 0 to exit.

References

Python Software Foundation. Python 3 Documentation. https://docs.python.org/3/ (accessed 24 October 2026).

OpenDSA Project. Data Structures and Algorithms Modules: Linked Lists, Queues, and Stacks. https://opendsa-server.cs.vt.edu/ (accessed 24 October 2026).

Course notes and classroom explanations for Data Structures and Algorithms, SGT University, academic session 2026-27.